

JECRC™
UNIVERSITY
BUILD YOUR WORLD

Prompt Engineering for C and C++
2026-2027

Course: B.Tech CSE (Core)
Registration number: 25BCON1594
Section: I
Semester: III
Submitted to: Mr. Ajay Sharma
Submitted by: Chehak Gupta

Session 07-AI AS A HINT, NOT AN ANSWER

Problem 1: Two Sum

Objective: Given an integer array and a target, return the indices of two numbers that add to the target. Use each element at most once.

1. Hint Log

(Note: Spend 10 minutes solving independently before using hints. Log any hints used below.)

Hint Level (l-4)	What I tried before asking	What the hint said	What I did next
Level 1	I tried iterating with one loop but couldn't figure out how to compare two different elements without repeating indices.	"Using two nested loops to check every pair is a perfectly reasonable and correct direction to start with."	I implemented the double for loop, making sure the inner loop starts at i + 1 so I don't use the exact same element twice.

2. Independent Code & Complexity Claim

My Claim:

- **Time Complexity:** $O(n^2)$
- **Space Complexity:** 0(1)

- **Justification:** I used a nested loop (a loop inside a loop) to check every single pair of numbers, which means it runs roughly $n \times n$ times in the worst case. I did not use any extra data structures to store values, so the space required is constant, or 0(1).

Python

```python
# Time Complexity: O(n^2)
# Space Complexity: O(1)

def twoSum(nums, target):
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []

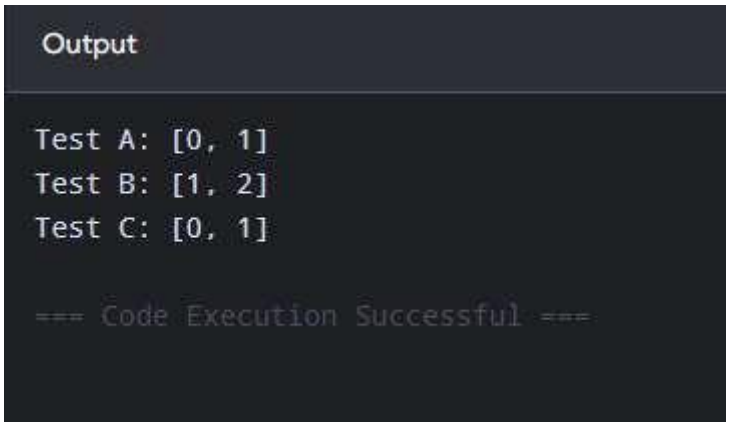
# Test Executions
print("Test A:", twoSum([2, 7, 11, 15], 9))
print("Test B:", twoSum([3, 2, 4], 6))
print("Test C:", twoSum([3, 3], 6))
```

3. Test *Case Executions*

Test A: [2, 7, 11, 15], target 9 (Expected: [0, 1])

Test B: [3, 2, 4], target 6 (Expected: [1, 2])

Test C (Edge Case - Duplicates): [3, 3], target 6 (Expected: [0, 1])



4. AI Review

Prompt Used:

Review this solution I wrote. Do not rewrite it.

1. List any edge cases it would fail on.

2. State its time and space complexity, and explain how you arrived at each.

3. Name one thing that could be clearer, without changing the approach.

AI Response:

1. **Edge Cases:** Your code is functionally correct and handles standard edge cases well (like duplicate numbers). However, if an array is massive (e.g., 100,000+ elements), your nested loops will cause a "Time Limit Exceeded" error in competitive programming environments. It also currently returns an empty list [] if no valid pair exists, rather than raising an exception.

2. Complexity:

a. **Time Complexity:** $O(n^2)$. The outer loop runs n times, and the inner loop runs up to n-1 times. This results in quadratic time execution.

b. **Space Complexity:** 0(1). You are evaluating the list in place and returning a small 2-element array, utilizing no scaling memory.

3. **Clarity:** Without changing the nested-loop approach, the code is actually very clear. However, using standard terminology like left and right forindices instead of i and j can sometimes make pointer-logic slightly more readable for reviewers.

5. Complexity Disagreement Resolution

No disagreement. The Al agreed with my assessment of $O(n^2)$ time and 0(1) space based on the nested loops and lack of extra storage.

Problem 2: Best Time to Buy and Sell Stock

Objective: Given daily prices, choose one day to buy and a later day to sell. Return the maximum profit, or O if no profit is possible.

1. Hint Log

Hint Level (1-4)	What I tried before asking	What the hint said	What I did next
Level 2	I tried nested loops to calculate the profit of every possible buy/sell combination, but realized it was too slow for large inputs and felt inefficient.	"A technique that is usually relevant here is keeping track of the minimum value seen so far while doing a single pass through the array."	I changed the code to only use one loop, updating min_price whenever I found a lower price, and updating max_profi t whenever the current potential profit was higher.

2. Independent Code & Complexity Claim

My Claim:

- **Time Complexity:** O(n)
- **Space Complexity:** 0(1)

- **Justification:** I used a single for loop to iterate through the prices list exactly once. I only stored two integer variables (min_price and max_profit), meaning memory does not scale with the input size, keeping space constant.

Python

```python
# Time Complexity: O(n)
# Space Complexity: O(1)

def maxProfit(prices):
    if not prices:
        return 0

    min_price = prices[0]
    max_profit = 0

    for price in prices:
        if price < min_price:
            min_price = price
        else:
            profit = price - min_price
            if profit > max_profit:
                max_profit = profit

    return max_profit

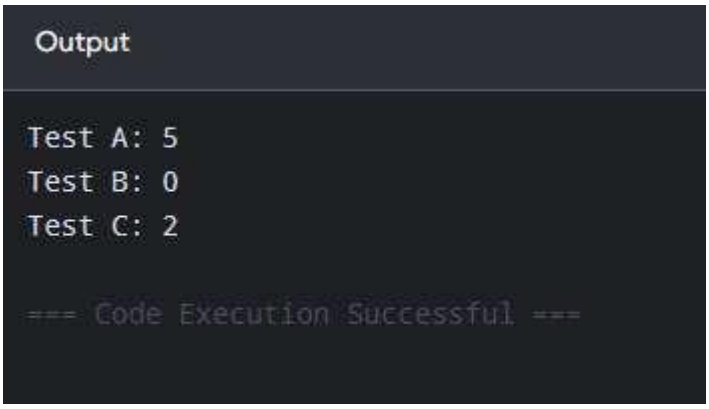
# Test Executions
print("Test A:", maxProfit([7, 1, 5, 3, 6, 4]))
print("Test B:", maxProfit([7, 6, 4, 3, 1]))
print("Test C:", maxProfit([2, 4, 1]))
```

3. Test Case Executions

Test A: [7,1,5,3,6,4] (Expected: 5)

Test B (Edge Case - Strictly Decreasing): [7, 6, 4, 3, 1] (Expected: 0)

Test C: [2,4,1] (Expected: 2)



4. AI Review

Prompt Used:

Review this solution I wrote. Do not rewrite it.

1. List any edge cases it would fail on.

2. State its time and space complexity, and explain how you arrived at each.

3. Name one thing that could be clearer, without changing the approach.

AI Response:

1. **Edge Cases:** Your solution is highly robust. It gracefully handles empty lists, lists with a single element (bypassing the loop logic successfully), and the strictly decreasing edge case (returning 0 because `profit > max_profit` is never triggered). There are no standard edge cases this would fail on.

2. Complexity:

a. **Time Complexity:** 0(n). You are performing a single pass over the array of prices.

b. **Space Complexity:** 0(1). You allocate memory for two basic variables (min_price and max_profit),which requires constant space regardless of input size.

3. **Clarity:** Your explicit if/else logic is very readable. To make it slightly cleaner without changing the approach, you could utilize Python'sbuilt-in max() function to update the profit (e.g., max_profit = max(max_profit, price - min_price)). This removes the need for the nested if statement.

5. Complexity Disagreement Resolution

No disagreement. Both the Al and I correctly identified the time complexity as 0(n) due to the single loop and the space complexity as 0(1) due to variable tracking.